

TUS OS® - Core Fixed Protocols & Technical Specification (Normative)

TUS OS®

Core Fixed Protocols & Technical Specification

Author: Mark Whitlock

Version: 1.0

Date: 8 March 2026

© 2026 Mark Whitlock. All rights reserved.

No use, reproduction, distribution, or modification is permitted without prior written permission from the copyright holder.

Trademark

TUS OS® is a trademark of Mark Whitlock. This publication does not grant any right to use the trademark except as permitted by applicable law. A detailed trademark and compatibility policy will follow.

TUS OS, referred to in this document as **TUS** where context is clear, is a protocol and reference architecture for turning a decision process into a finalized, self-verifying record that can be independently inspected and validated. Rather than treating decisions as ephemeral outputs, TUS produces canonical “receipts” that capture the state evaluated, the set of permitted moves considered, the scoring and tie-break rules applied, and the exact transition selected. The result is a deterministic, content-addressable artifact: given the same configuration and inputs, the same receipt is produced, and any third party can verify its integrity without rerunning the engine.

Operationally, TUS v1 represents a situation using a fixed **6-bit state space (64 states)** and evaluates local alternatives according to declared policy weights and neighborhood rules. In Guidance mode, it selects the best next adjustment within that constrained neighborhood and records not only the choice but the ranked comparisons that led to it. In Landing-style modes, it treats the selected transition as a descriptive projection spine—describing how pressure may resolve as motion occurs—while enforcing strict guardrails that prevent outcome promises, invented context, or covert recommendations. Interpretation is domain-bound and receipt-grounded: the canonical receipt is the authoritative description of the state, and any narrative layer must remain faithful to declared fields and explicitly name ambiguity rather than filling gaps.

TUS matters because it provides a missing primitive for accountable decision systems: the ability to bind analysis and action to a verifiable, immutable decision artifact. In environments where trust, compliance, and dispute resolution dominate—finance, insurance, public-sector allocation, enterprise governance, and AI oversight—post-hoc explanations and opaque model outputs are increasingly insufficient. TUS makes decisions reproducible, auditable, and defensible by construction. Used as a governance layer around AI, it enables exploration upstream while ensuring that what gets committed downstream is constrained, testable, and provably derived from declared rules—turning “because the model said so” into “here is the exact, verifiable basis on which this conclusion was reached.”

Status: v1.0 — Normative Core Protocol

This specification defines the fixed, normative protocol surface for **TUS Core** within the broader **TUS OS** stack and the minimum conditions for protocol conformance, receipt validity, and independent verification.

This document defines the **fixed, deterministic protocols** that govern TUS behavior across implementations.

Relationship to the White Paper

- The **TUS OS White Paper** is informational.
- This specification defines the normative core protocol for **TUS Core**.
- Where wording differs, this specification governs protocol conformance.
- Where this specification explicitly references a separately versioned companion specification, that companion specification is authoritative only for the surface it defines.
- For `LANDING_TRIAD_SET_V1`, the separately versioned **TUS LANDING_TRIAD_SET Specification v1.0** is a normative companion specification. Variant-specific orchestration semantics for that set are governed by that companion specification.

These rules are **normative** and MUST NOT vary without an explicit version bump.

For clarity: protocol conformance means adherence to the fixed rules defined by this specification; receipt validity means that an emitted receipt is complete, internally consistent, and verifiable under the declared receipt protocol; implementation guidance means non-authoritative advice about how to realize the protocol without changing canonical outcomes.

These companion layers are intentionally kept outside the receipt core to avoid bloat while still permitting independent semantic verification when paired with the appropriate policy pack and companion specifications.

This specification consolidates the fixed protocol rules implemented across the core code surfaces:

- tus-core
- tus-math
- tus-receipt
- tus-sets

It is written from an audit, verifier, and implementer perspective.

This specification proceeds from architectural invariants, to determinism and math rules, to state and scoring semantics, to receipt authority, to set orchestration, and finally to verifier obligations.

TUS Stack Position (Normative)

This document defines the **Core Protocol** only:

- deterministic kernel behavior
- receipt protocol
- set and orchestration protocol

Companion specifications will define the normative protocols for Domain Binding and Output Protocol behavior; those layers are intentionally outside the scope of this core specification.

This document does not define the full **TUS OS** stack; it defines only the fixed protocol surface required for **TUS Core** conformance.

The following are separate companion layers and are not defined by this specification:

- Domain Binding / Domain Profiles
- evaluation rubric(s)
- linting / output validation
- narrative rendering / explanatory policy

Where Domain Binding / Domain Profiles are used, they are treated as declared, receipt-bound inputs and **MUST NOT** modify kernel computation, ranking, selection, or receipt integrity.

Normative Scope and Non-Goals

In Scope

This specification defines:

- deterministic decision semantics
- numeric and ordering guarantees
- selection, ranking, and anchoring rules
- receipt integrity and verification boundaries
- set-level orchestration rules that affect outcomes

Explicit Non-Goals

This specification does **not**:

- define domain meaning or interpretation
- guarantee outcome optimality
- require rerunning the decision engine
- specify rendering, visualization, or explanation formats
- freeze UI, narrative, or presentation layers

Glossary (Normative)

- **Protocol**: A fixed, versioned set of rules that affect canonical outcomes.
- **Specification (Spec)**: A normative document that defines a protocol.
- **Policy Pack**: A content-addressed bundle of normative artifacts bound by a manifest.
- **Artifact**: A hash-addressed object (text or data) referenced by a Policy Pack.
- **Receipt**: The canonical, immutable record of a single decision.
- **View**: A canonical projection of an object used for hashing and verification.

Conformance note: The white paper refers to “Tier A vs Tier B” at a conceptual level. This specification defines the hardened Tier A protocol only. Tier B was a Tier A hardening phase and is not defined by this document.

1. Architectural Separation (Normative)

TUS is defined as four strictly separated layers:

1. **Kernel / Math Layer** – deterministic numeric primitives and reduction order
2. **Decision Kernel** – state, neighborhood, scoring, ranking, selection
3. **Receipt Protocol** – canonical serialization, hashing, validation, sealing
4. **Set & Orchestration Layer** – multi-run structure (e.g., triads), anchor policy

Rules in layers (1)–(4) are **fixed protocols**. Narrative interpretation, rendering, and domain meaning are explicitly non-normative.

2. Determinism Contract (Global, Normative)

2.1 Determinism Scope

Given identical:

- input state(s)
- engine/kernel version identifiers
- math backend identifiers
- policy/spec identifiers

A conformant TUS implementation **MUST** produce:

- identical canonical receipts
- identical hash projections
- identical selections and rankings

No implementation-defined defaults are permitted.

2.2 Prohibited Sources of Variance

Interpretation, rendering, domain meaning, and downstream narrative **MUST NOT** modify scoring, ranking, selection, or receipt integrity. This preserves the one-way contract from computation to receipt to interpretation.

The following are forbidden in conformant execution paths:

- non-deterministic iteration order
- parallel or associative reductions
- platform math intrinsics

- fast-math / reassociation / FMA contraction
 - locale-dependent serialization
-

3. Numeric & Math Protocol (tus-math) – Fixed

3.1 Floating-Point Environment

Conformant execution MUST use:

- IEEE-754 binary64
- round-to-nearest, ties-to-even
- subnormals preserved (FTZ/DAZ disabled)
- no implicit widening or narrowing

3.2 Canonical Math Backend

All transcendental operations (`exp64` , `ln64` , `sqrt64`) MUST be:

- provided by a shipped, versioned backend
- identical across platforms
- referenced by a `math_backend_id`

3.3 Reduction Order

All summations and reductions MUST be:

- single-threaded
- performed in ascending, declared index order
- non-compensated

This rule is critical for bitwise reproducibility.

4. State Space & Neighborhood Protocol (tus-core) – Fixed

4.1 State Representation

- For this specification, state is a fixed **6-bit** bitfield.
- Line indices are ordered **1..6**.
- Bit significance and ordering are fixed and versioned.
- Any kernel with a different dimension requires a separate versioned kernel/protocol specification and is not conformant to this specification.

4.2 Neighborhood Definition

For a state h , the neighborhood consists of exactly **six** states:

- $h_i = h$ with line i flipped, for $i \in \{1..6\}$

No other moves are permitted for this kernel specification version. Set variants MAY further restrict permitted moves only if explicitly declared and recorded in the receipt or set manifest.

5. Canonical Scoring Kernel – Fixed

5.1 Admitted Upstream Quantity

Exactly one upstream scalar is permitted: $\text{frag}(h)$ – a deterministic, state-only quantity.

In this specification, $\text{frag}(h)$ is defined by the separately versioned **Entropy Kernel Specification**, which is authoritative for its derivation when referenced.

This specification does not restate that procedure.

5.2 Weight and Probability Construction

For each neighbor h_i :

1. $w_i = |\text{frag}(h_i) - \text{frag}(h)| + \epsilon_{\text{scale}}$
2. $q_i = w_i / \sum w_j$
3. $\phi_i = -q_i * \ln_{64}(q_i)$
4. $\Phi = \sum \phi_i$
5. Center: $c_i = \phi_i - \text{mean}(\phi_i)$
6. Scale: $T = \text{std_pop}(c) + \epsilon_{\text{scale}}$
7. Softmax input: $x_i = c_i / T$

All steps MUST follow the declared evaluation order.

5.3 Softmax Protocol

- Compute $m = \max(x_i)$ (ties $\rightarrow$ lowest index)
- Compute $e_i = \exp_{64}(x_i - m)$ in ascending index order
- Compute $z = \sum e_i$ in ascending index order
- $p_i = e_i / z$

$p_{\text{line}}(h)$ is the ordered vector $\{p_1 \dots p_6\}$.

5.4 Uniform Neighborhood Predicate

Neighborhood is uniform iff all `w_i` are **exactly equal in binary**⁶⁴. No tolerance or epsilon comparison is allowed.

6. Ranking, Selection, and Tie-Break Protocol – Fixed

6.1 Ranking Rule

Rank neighbors by the tuple:

1. descending `p_i`
2. ascending line index

This defines a total order.

6.2 Guidance Selection

In `GUIDANCE` mode:

- Select `i* = argmax(p_i)`
- Apply deterministic tie-break (lowest index)
- Transition is `h → h_{i*}`

No randomness is permitted.

7. RNG Authority & Mode Separation – Fixed

7.1 Mode Definitions

- `CAST` – the canonical initial-state selection protocol, defined by the separate CAST Specification. CAST is the sole authority for establishing `h0` prior to downstream execution and applies across modes.
- `GUIDANCE` – fully deterministic transition selection after CAST.
- `LANDING` – post-CAST landing realization semantics; stochastic variants MUST declare replayable evidence, and deterministic variants MUST declare no such RNG.

In canonical receipts, execution mode is recorded separately from CAST evidence. CAST governs initial-state selection and replay evidence; GUIDANCE and LANDING govern downstream transition behavior.

7.2 RNG Authority Rules

- CAST randomness MUST be declared, replayable, and singular.
- CAST semantics, evidence requirements, and replay rules are governed by the separate CAST Specification and are not restated here.
- No hidden or secondary randomness is allowed.

- In `GUIDANCE`, no randomness is permitted after CAST.
- In `LANDING`, any post-CAST stochastic operation **MUST** be explicitly declared and recorded with replayable evidence.
- `LANDING_TRIAD_SET_V1` is a deterministic post-CAST variant: after the initial anchor state, no RNG is permitted for LANDING line selection or lookahead.
- CAST randomness **MUST NOT** be reused to bias downstream selection unless explicitly allowed by a separate normative specification.

8. Receipt Protocol (tus-receipt) – Fixed

8.0 Authority Hierarchy (Normative)

In case of conflict:

- The **canonical receipt** is authoritative for what occurred.
- Fixed protocol specifications are authoritative for what was permitted.

8.1 Canonical Receipt

The canonical receipt is the sole authoritative artifact for a realized decision event. It **MUST**:

- fully describe the evaluated state
- enumerate permitted moves
- record scoring outputs and rankings
- record the selected transition

A canonical receipt may contain both a hashed canonical view and a fuller receipt envelope. For integrity verification, the authoritative hashed projection is `RECEIPT_VIEW_V1`. Additional packaging, provenance, set, or auxiliary fields **MAY** exist outside that hashed view, but they do not affect the Tier A canonical hash unless explicitly included by the declared projection.

8.2 Canonical Serialization

- UTF-8
- lexicographic object keys
- stable array order
- no insignificant whitespace

Numeric values that are hash-driving receipt fields **MUST** be encoded using the canonical dual representation `{ "v": <binary64 value>, "s": <canonical string form> }` where required by the receipt schema.

8.3 Hash Projections

Receipts MUST expose at minimum:

- `CORE_VIEW_V1`
- `RECEIPT_VIEW_V1`

Hashes are computed as:

```
SHA256(canonical_bytes(view))
```

Self-referential and non-normative fields are excluded by definition.

For strict field-level validator conformance, the authoritative source of truth for `RECEIPT_VIEW_V1` is the emitted audit pointer map for the declared projection.

8.4 Integrity Guarantees

This specification asserts:

- immutability
- internal consistency
- deterministic completeness
- verifiable integrity without rerunning the engine, except where a separate verifier profile explicitly requires recomputation

9. Set & Orchestration Protocol (tus-sets) – Fixed

9.1 Set Semantics

Sets coordinate multiple independent receipts under a declared variant. Set logic MUST NOT alter per-receipt scoring semantics.

Set execution MAY emit auxiliary set-level artifacts, including canonical set records, foundation records, synthesis records, and manifests. Such artifacts do not redefine per-receipt core semantics unless explicitly declared by a normative companion specification.

9.2 Triad Construction

- Start A chosen by the declared CAST protocol
- Starts B and C derived to maximize minimum pairwise Hamming distance
- Deterministic tie-breaks apply

9.3 Landing Anchor Mapping (Triad Variant)

Within a triad:

- Pair 1 → RUNNER-UP rank
- Pair 2 → MEDIAN rank
- Pair 3 → LOWEST rank

No RNG is permitted for anchor selection in this variant.

Variant-specific receipt semantics for `LANDING_TRIAD_SET_V1` — including `chosen_line_step1`, `chosen_line_step2`, `selection.to_state`, and `landing_to_state` — are governed by the normative companion triad specification.

9.4 Triad Structural Purpose (Normative)

`LANDING_TRIAD_SET` is designed to expose structural properties of the canonical decision field by sampling ranked transition coordinates across maximally separated starting states.

Accordingly:

- triad construction provides directional coverage of the bounded state space rather than redundant repetition,
- `GUIDANCE` exposes the dominant ranked transition at each start,
- anchor-based `LANDING` exposes non-dominant ranked transition coordinates from the same canonical field,
- set-level orchestration MUST NOT alter per-receipt scoring, ranking, or neighborhood semantics.

This variant therefore measures structural variation across state-space position and rank position while preserving a single shared kernel and a single shared scoring geometry.

`LANDING_TRIAD_SET` is an additive set-level orchestration over the shared canonical kernel; it does not redefine the kernel, the receipt protocol, or downstream semantics.

For conformance and verification, authority order is: this fixed protocol specification for core protocol rules; the canonical receipt for what actually occurred in a realized decision event; and separately versioned companion specifications for variant-specific set behavior where explicitly referenced.

10. External Bindings (Limited Scope)

10.0 Authority Rule

Only the `pack_digest` is authoritative. `pack_id` and `pack_version` are descriptive and MUST NOT be used for validation or trust decisions.

10.1 Limited Role

External bindings do not alter core protocol behavior unless a separate normative specification explicitly says so.

10.2 Permitted Use

External bindings MAY identify companion artifacts such as:

- scoring kernel specifications
- anchor mapping policies
- fixed ordering or tie-break tables
- math backend source or verification material

10.3 Verification Boundary

Core verification depends on:

- the canonical receipt
- this fixed protocol specification

If a receipt declares external dependencies, those dependencies MAY be inspected, but they are not part of core receipt integrity verification unless explicitly declared by a separate normative specification.

11. Verifier Minimal Obligations (Normative)

11.1 Verifier MUST

A conformant verifier MUST, at minimum:

1. Recompute and verify `CORE_VIEW_V1` hash
2. Recompute and verify `RECEIPT_VIEW_V1` hash
3. Verify declared `math_backend_id` and FP contract
4. Verify that any stochastic operation permitted by the declared mode/specification is explicitly declared and evidenced, and that no hidden or forbidden randomness is present
5. Verify receipt finalization and immutability flags

12. Downstream Layers and Out-of-Scope Semantics

The following MUST NOT affect canonical outcomes:

- narrative interpretation
- visualization or rendering
- domain labeling
- explanatory prose

These layers MUST be receipt-grounded and ambiguity-preserving.

Change Control and Versioning (Normative)

Changes That REQUIRE a New Fixed Protocol Version

- any change to math backend behavior or precision
- any change to evaluation or reduction order
- any change to tie-break or ordering rules
- any change to neighborhood or move legality
- any change to anchor mapping or set orchestration semantics

Changes That Do NOT Require a New Version

- renderer or visualization changes
- narrative or explanatory text
- documentation wording that does not alter semantics
- UI or presentation-layer modifications

Explicit Surfacing of Implicit Policies

This specification intentionally surfaces policies that are often left implicit in decision engines, including:

- numeric reduction order
- tie-break resolution
- iteration and ordering guarantees
- placement and authority of randomness

These rules are fixed here to prevent silent divergence across implementations.

Policy Pack Manifest Schema

The machine-readable Policy Pack Manifest Schema is maintained as a separate companion specification.

This core protocol specification does not restate that schema.

Where used, Policy Pack manifests MUST be bound by `pack_digest` and validated against the separately versioned Policy Pack Manifest Schema.

End of Fixed Protocol Specification